Roll No: _____ **Date:** _____

Experiment-6
IPC (Inter-Process Communication)

Theory

Definition

IPC is a mechanism that allows processes (independent programs running in an operating system) to communicate and share data with each other.

Purpose

Since processes have separate memory spaces, IPC provides controlled ways to exchange information, synchronize actions, and coordinate tasks.

Examples of IPC methods:

- Pipes
- Message Queues
- Shared Memory
- Semaphores
- Sockets

Shared Memory

- **Definition:** Shared memory is an IPC technique where a memory region is mapped so that multiple processes can access it directly.
- **Key Point:** Since all processes can read/write the same memory region, it is the fastest form of IPC.
- **Drawback:** Requires synchronization (e.g., semaphores) to avoid race conditions (two processes writing/reading at the same time).

Semaphore

- **Definition:** A semaphore is a synchronization tool used to control access to shared resources by multiple processes.
- **Types:**

  - **Binary Semaphore (Mutex):** Can be 0 or 1, used for mutual exclusion (only one process can enter the critical section at a time).
  - **Counting Semaphore:** Can have values > 1, used to manage a resource pool (like multiple identical printers).

- **Use Case:** Prevents race conditions and ensures process synchronization when using shared resources (like shared memory).

Pipe

- **Definition:** A pipe is an IPC mechanism that allows data to flow in one direction between processes. It acts like a buffer inside the kernel with two ends:
  - **Write End:** one process writes data.
  - **Read End:** another process reads data.

- **Types of Pipes:**
  1. **Anonymous Pipe** - created using `pipe()`, used for communication between parent and child processes.
  2. **Named Pipe (FIFO)** - created using `mkfifo()`, allows communication between unrelated processes.

Roll No: _____ **Date:** _____

Program 1: Anonymous Pipe

Aim: Write a program to demonstrate a pipe concept.

Description

This program demonstrates Inter-Process Communication (IPC) between a parent and child process using a pipe. A pipe is a unidirectional communication channel that allows one process to write data while another reads it.

Here, the parent process writes the message "Hello from parent via pipe" into the pipe, and the child process reads it. Before communication, each process closes the unused end of the pipe to avoid deadlock:

- Parent closes the read end and writes to the pipe.
- Child closes the write end and reads from the pipe.

Thus, the parent and child successfully exchange data using a pipe, proving how IPC works in Unix/Linux systems.

Algorithm

1. Start.
2. Include required header files (`stdio.h`, `unistd.h`, `string.h`, `stdlib.h`).
3. Declare a pipe using `int fd[2]`, where `fd[0]` is the read end and `fd[1]` is the write end.
4. Create a pipe using `pipe(fd)`. If it fails, print an error and exit.
5. Create a new process using `fork()`. If `fork()` fails, print an error and exit.
6. If `pid > 0` (Parent process):
   - Close the read end (`fd[0]`).
   - Write a message into the pipe using `write(fd[1], ...)`.
   - Close the write end after writing.
7. If `pid == 0` (Child process):
   - Close the write end (`fd[1]`).
   - Read the message from the pipe using `read(fd[0], ...)`.
   - Print the received message.
   - Close the read end.
8. End.

Source Code

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

int main() {
    int fd[2]; // fd[0] = read end, fd[1] = write end
    pid_t pid;
    char write_msg[] = "Hello from parent via pipe!";
    char read_msg[100];

    // Create a pipe
    if (pipe(fd) == -1) {
        perror("Pipe failed");
        exit(1);
    }

    // Create child process
    pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    if (pid > 0) {
        // Parent process
        close(fd[0]); // close read end
        printf("Parent writing: %s\n", write_msg);
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]); // close write end
    } else {
        // Child process
        close(fd[1]); // close write end
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[0]); // close read end
    }

    return 0;
}
```

Output

```
student@cselab:~$ gcc threads.c
student@cselab:~$ ./a.out
Parent writing: Hello from parent via pipe!
Child received: Hello from parent via pipe!
```

Roll No: _____ **Date:** _____

Program 2: Producer - Consumer Problem using Shared Memory

Aim: Write a program to demonstrate Producer - Consumer Problem using Shared Memory.

Description:

This program demonstrates interprocess communication (IPC) using POSIX shared memory and semaphores for synchronization. It solves the classical Producer–Consumer problem where:

- Producer generates items (numbers 1–10) and places them in a bounded buffer.
- Consumer takes items from the buffer in the same order and processes them.
- Shared Memory is used as the buffer accessible by both processes.
- Semaphores are used to prevent race conditions:
  - mutex ensures mutual exclusion while accessing the buffer.
  - empty ensures producer waits if the buffer is full.
  - full ensures consumer waits if the buffer is empty.

The buffer size is fixed (BUFFER_SIZE = 8), so synchronization ensures that neither process overflows or underflows the buffer.

At the end, resources (semaphores and shared memory) are released properly.

Algorithm

1. **Create Shared Memory Segment**
   - Use `shm_open()` to create/open a POSIX shared memory object.
   - Resize it with `ftruncate()` to fit the shared structure.
   - Map it into the process address space using `mmap()`.
2. **Initialize Shared Memory Structure**
   - Set `in = 0`, `out = 0`.
   - Initialize semaphores: `mutex = 1` (mutual exclusion), `empty = BUFFER_SIZE` (counts empty slots), `full = 0` (counts filled slots).
3. **Fork Process**
   - Parent process → Producer.
   - Child process → Consumer.
4. **Producer Process**

- Repeat for each item to produce:
- Wait (`sem_wait(&empty)`) until there is at least one empty slot.
- Lock the buffer (`sem_wait(&mutex)`).
- Insert item at position `in`.
- Move `in = (in + 1) % BUFFER_SIZE`.
- Unlock (`sem_post(&mutex)`).
- Signal (`sem_post(&full)`) that a new item is available.

5. **Consumer Process**

- Repeat for each item to consume:
- Wait (`sem_wait(&full)`) until there is at least one full slot.
- Lock the buffer (`sem_wait(&mutex)`).
- Remove item at position `out`.
- Move `out = (out + 1) % BUFFER_SIZE`.
- Unlock (`sem_post(&mutex)`).
- Signal (`sem_post(&empty)`) that a slot is free.

6. **Termination & Cleanup**

- Parent waits for child using `waitpid()`.
- Destroy semaphores (`sem_destroy`).
- Unmap and unlink shared memory (`munmap`, `shm_unlink`).
- Print cleanup confirmation.

Source Code

```c
#define _XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <errno.h>
#include <string.h>
#include <time.h>
#include <sys/wait.h>

#define SHM_NAME "/po_shm_example"
#define BUFFER_SIZE 8
#define ITEMS_TO_PRODUCE 10

typedef struct {
    int buffer[BUFFER_SIZE];
    int in;
```

```c
    int out;
    sem_t mutex; // binary semaphore for mutual exclusion
    sem_t empty; // counts empty slots
    sem_t full;  // counts filled slots
} shared_t;

static void die(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

static void producer(shared_t *sh) {
    for (int item = 1; item <= ITEMS_TO_PRODUCE; ++item) {
        // Wait for an empty slot, then lock the buffer
        if (sem_wait(&sh->empty) == -1) die("sem_wait empty");
        if (sem_wait(&sh->mutex) == -1) die("sem_wait mutex");

        // Critical section: put item
        sh->buffer[sh->in] = item;
        int slot = sh->in;
        sh->in = (sh->in + 1) % BUFFER_SIZE;

        // Unlock and signal that a new full slot is available
        if (sem_post(&sh->mutex) == -1) die("sem_post mutex");
        if (sem_post(&sh->full) == -1) die("sem_post full");

        printf("[Producer] Produced %d at slot %d\n", item, slot);
        fflush(stdout);
    }
}

static void consumer(shared_t *sh) {
    for (int count = 0; count < ITEMS_TO_PRODUCE; ++count) {
        // Wait for a full slot, then lock the buffer
        if (sem_wait(&sh->full) == -1) die("sem_wait full");
        if (sem_wait(&sh->mutex) == -1) die("sem_wait mutex");

        // Critical section: get item
        int item = sh->buffer[sh->out];
        int slot = sh->out;
        sh->out = (sh->out + 1) % BUFFER_SIZE;

        // Unlock and signal that an empty slot is available
        if (sem_post(&sh->mutex) == -1) die("sem_post mutex");
        if (sem_post(&sh->empty) == -1) die("sem_post empty");

        printf("[Consumer] Consumed %d from slot %d\n", item, slot);
        fflush(stdout);
    }
}

int main(void) {
    // Create or open the shared memory object
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (fd == -1) die("shm_open");

    // Resize to fit our shared struct
```

```
    if (ftruncate(fd, sizeof(shared_t)) == -1) die("ftruncate");

    // Map it into our address space (shared, read/write)
    void *addr = mmap(NULL, sizeof(shared_t), PROT_READ | PROT_WRITE,
   MAP_SHARED, fd, 0);
    if (addr == MAP_FAILED) die("mmap");
    close(fd); // fd no longer needed after mmap

    shared_t *sh = (shared_t *)addr;

    // Initialize shared region (only once, before fork)
    sh->in = 0;
    sh->out = 0;
    if (sem_init(&sh->mutex, 1, 1) == -1) die("sem_init mutex");
    if (sem_init(&sh->empty, 1, BUFFER_SIZE) == -1) die("sem_init empty
   ");
    if (sem_init(&sh->full, 1, 0) == -1) die("sem_init full");

    pid_t pid = fork();
    if (pid == -1) die("fork");

    if (pid == 0) {
        // Child: consumer
        consumer(sh);
        exit(0); // Child done
    } else {
        // Parent: producer
        producer(sh);

        // Wait for child to finish
        int status = 0;
        if (waitpid(pid, &status, 0) == -1) die("waitpid");

        // Cleanup semaphores
        if (sem_destroy(&sh->mutex) == -1) die("sem_destroy mutex");
        if (sem_destroy(&sh->empty) == -1) die("sem_destroy empty");
        if (sem_destroy(&sh->full) == -1) die("sem_destroy full");

        // Unmap and unlink shared memory
        if (munmap(sh, sizeof(shared_t)) == -1) die("munmap");
        if (shm_unlink(SHM_NAME) == -1) die("shm_unlink");

        printf("\nAll done. Cleaned up shared memory and semaphores.\n"
   );
    }
    return 0;
}
```

Output

```
student@cselab:~$ gcc producer_consumer_shm.c -lpthread -lrt
student@cselab:~$ ./a.out
[Producer] Produced 1 at slot 0
[Producer] Produced 2 at slot 1
[Producer] Produced 3 at slot 2
```

```
[Producer] Produced 4 at slot 3
[Producer] Produced 5 at slot 4
[Producer] Produced 6 at slot 5
[Producer] Produced 7 at slot 6
[Producer] Produced 8 at slot 7
[Producer] Produced 9 at slot 0
[Consumer] Consumed 1 from slot 0
[Consumer] Consumed 2 from slot 1
[Consumer] Consumed 3 from slot 2
[Producer] Produced 10 at slot 1
[Consumer] Consumed 4 from slot 3
[Consumer] Consumed 5 from slot 4
[Consumer] Consumed 6 from slot 5
[Consumer] Consumed 7 from slot 6
[Consumer] Consumed 8 from slot 7
[Consumer] Consumed 9 from slot 0
[Consumer] Consumed 10 from slot 1

All done. Cleaned up shared memory and semaphores.
```

Program 3: Named Pipe

Aim: Write a program to demonstrate named pipe.

Description

This program demonstrates Inter-Process Communication (IPC) using a Named Pipe (FIFO).

- `Npc.c`: Creates a persistent FIFO special file in the filesystem. This acts like a "mailbox" for communication.
- `Sender.c`: Opens the FIFO in write-only mode, accepts user input, and writes the message into the FIFO.
- `Receiver.c`: Opens the FIFO in read-only mode, reads the data, and displays the message.

Since the FIFO exists in the filesystem, multiple processes can use it for one-way communication. It is half-duplex: data flows in only one direction at a time (sender $\rightarrow$ receiver).

Algorithm

1. **Step 1: Create FIFO (`Npc.c`)**
   (a) Use `mkfifo("fifo", 0777)` to create a named pipe.
   (b) If it already exists, it can be reused.
2. **Step 2: Sender Process (`Sender.c`)**
   (a) Open FIFO in write-only mode.
   (b) Read a string from user input.
   (c) Write the string into the FIFO.
   (d) Close the FIFO.
3. **Step 3: Receiver Process (`Receiver.c`)**
   (a) Open FIFO in read-only mode.
   (b) Read the message from FIFO.
   (c) Print the received message.
   (d) Close the FIFO.

Source Code

Npc.c

```c
#include <stdio.h>
#include <sys/stat.h>

int main() {
    mkfifo("fifo", 0777); // create named pipe with full permissions
    printf("Named pipe created\n");
    return 0;
}
```

Sender.c

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char msg[100];

    fd = open("fifo", O_WRONLY);
    printf("Enter a message: ");
    fgets(msg, sizeof(msg), stdin);
    write(fd, msg, sizeof(msg));

    close(fd);
    return 0;
}
```

Receiver.c

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char msg[100];

    fd = open("fifo", O_RDONLY);
    read(fd, msg, sizeof(msg));
    printf("Received message: %s", msg);

    close(fd);
    return 0;
}
```

Output

```
student@cselab:~$ gcc npc.c -o npc
```

```
student@cselab:~$ ./npc
Named pipe created

student@cselab:~$ gcc sender.c -o sender
student@cselab:~$ ./sender
Enter a message: This is named pipe test

student@cselab:~$ gcc receiver.c -o receiver
student@cselab:~$ ./receiver
Received message: This is named pipe test
```